

Product Spec: Claude Project Sync v7

Product Overview

Product Name: Claude Project Sync (claudesync2)

Problem Statement: Developers using Claude Code (CLI) work across many projects, but Claude.ai (web) has no visibility into project status, progress, or recent work. Each new conversation starts from zero context. Chat history contains valuable project intelligence that's trapped in export zips. There's no bridge between the two environments.

Solution: A local-first system that:

- Maintains per-project status files (PROJECT_STATUS.md)
- Runs a live web dashboard with real-time project data
- Parses Claude.ai chat history exports and matches conversations to initiatives
- Automatically enriches and classifies using the Anthropic API when a key is present
- Generates a downloadable PDF report for sharing with Claude.ai
- Tracks open threads, blockers, and decisions across all projects
- Uses AI to classify unmatched conversations into new or existing initiatives

Architecture

```
PROJECT_STATUS.md files (per-project, in each repo)
|
v
generate_status_pdf.py — scanner, parser, chat matcher, PDF renderer
|
v
web_app.py — Flask dashboard (localhost:5111)
|
■■■ Dashboard (stats, chart, project cards, filters)
■■■ Drill-down views (/view/projects, /events, /threads, /blockers)
■■■ What's Next triage (/view/whats-next)
■■■ Unmatched conversations (/view/unmatched)
■■■ Project detail pages (/project/<idx>)
■■■ Upload modal (chat history zip + scan paths)
■■■ Background processing (threading + progress polling)
■■■ AI integration (auto when key present, prompt when not)
■■■ PDF generation (/generate-pdf)
```

Persistence layer (all `~/.claudesync_*.json`):

- `_ignored.json` — hidden/EOL project names
- `_resolved.json` — resolved thread keys
- `_archived.json` — archived project names
- `_ai.json` — API config (url, key, model)
- `_import_meta.json` — last chat import timestamp + filename
- `_item_actions.json` — item ignore/reassign decisions (What's Next)
- `_initiatives.json` — AI classification results for unmatched chats

User Story

Who: A developer managing 5-30+ projects, using Claude Code for development and Claude.ai for planning/research.

Workflow:

Developer works in Claude Code — commits happen on `claude/*` branches, `PROJECT_STATUS.md` files get updated

Developer opens `http://localhost:5111` in their browser

Clicks "Upload Chat History" — drops in a Claude.ai export zip + scan paths

If API key exists: AI enrichment + initiative classification runs automatically in the background

If no API key: A prompt appears asking "Enable AI-Enhanced Results?" with an API key field — OK to save and enable, Skip to proceed without

App stays fully usable during processing — top-nav progress bar shows step-by-step status

Dashboard populates with all projects, activity charts, thread counts, blockers

Clicks into any project for full timeline and open threads

Checks "What's Next" for cross-project triage of all action items

Checks "Unmatched Conversations" to see chat that couldn't be auto-matched + AI suggestions

Clicks "Generate PDF" for a downloadable report to share with Claude.ai

Core Functionality

1. Project Status Files (`PROJECT_STATUS.md`)

Each project directory contains a `PROJECT_STATUS.md` with:

- Metadata table (name, repo, category, progress %, status, last worked, last synced, dev state)
- "What's Working" bullet list
- "What's Not Working" bullet list
- "Blockers" bullet list
- "Progress Log" with dated session entries (### YYYY-MM-DD)
- "Open Design Questions" markdown table
- "Next Steps" numbered/bulleted list
- "Reusable Assets" table

Critical Design Decision: `CLAUDE.md` contains ONLY behavioral instructions for Claude Code. Status data belongs ONLY in `PROJECT_STATUS.md`. These must never be mixed.

2. Web Dashboard (`web_app.py`)

Flask app running on `localhost:5111`. Fully local — nothing leaves the machine.

Dashboard page (/):

- 5 clickable stat cards: Projects, Events, Threads, Blockers, Unmatched Chats — each links to a drill-down view
- Stacked bar chart (Chart.js) showing activity across all projects, toggleable by Day/Week/Month
- Filter bar: Active, Possibly EOL, Ignored
- Project card grid (responsive, 320px min) with:
 - Progress bar (color-coded by %)
 - Activity counts (commits, chats, logs)
 - Last 3 decisions
 - First 3 open threads with color-coded tags

- EOL detection with Ignore button
- Dark/light theme toggle (persisted in localStorage)
- Top-bar buttons: What's Next, Upload Chat History, AI Settings, Generate PDF
- Upload Chat History modal
- AI Settings modal (manual override for url/key/model)

Drill-down list views:

- /view/projects — all projects, split into Active and Archived sections, with Archive/Unarchive buttons
- /view/events — all timeline events across all projects, newest first, tagged by type (Commit/Chat/Log)
- /view/threads — all open threads with Resolve buttons, grouped by project
- /view/blockers — filtered to blocker-type threads only, with Resolve buttons
- /view/whats-next — cross-project triage (see Section 9)
- /view/unmatched — unmatched chat conversations + AI classification (see Section 11)

Project detail page (/project/):

- Header card with metadata, progress bar, narrative description
- Dev state dropdown (test/refine/continue) with color-coded badge
- "AI Assess" button to auto-classify dev state using AI
- Full decision timeline with dated entries, source tags, colored bullets
- Open threads card with Resolve buttons
- "Enrich with AI" button (when API key configured)

3. Non-Blocking Background Processing

All heavy work (scanning, parsing, chat matching, AI enrichment, AI classification) runs in a **background thread**. The app stays fully usable during processing.

Top-nav progress bar:

- Slim bar below the top bar, visible only during processing
- Shows spinner + step name + detail message
- Steps: Scanning → Parsing → Reading Chat → Matching Chat → AI Enrichment → AI Classifying → Building → Done
- Progress track fills based on current step
- Auto-reloads the page 1.5s after completion
- Error state shown in red if something fails

Implementation: Background thread + /api/progress polling every 500ms. No SSE or WebSockets needed.

4. Chat History Integration

Upload flow:

- User exports chat history from Claude.ai (zip file)
- Uploads via drag-and-drop or file picker in the Upload modal
- System extracts .md/.txt/.json files from the zip
- Two-tier matching:

Per-conversation: If ANY paragraph in a chat file mentions a project, that whole file counts as a conversation event for that project

Per-paragraph: Best-scoring paragraphs become detailed timeline entries with excerpts

- Import timestamp and filename saved for reference

Matching algorithm:

- Builds search terms per project: name variants, without version, squashed, first words, longest word
- Scores paragraphs by: term frequency * term length, length bonus, descriptive word bonus
- Deduplicates by word overlap ratio (>0.65 = skip)
- Keeps up to 50 snippets per project
- Minimum paragraph length: 30 characters
- Code ratio filter: skip if >60% code lines

Unmatched tracking:

- Chat files that don't match any project are collected with excerpts
- Stored in chat stats for the AI classification step
- Shown on the dashboard as an "Unmatched Chats" stat card

5. AI Integration

Automatic behavior:

- **When API key exists:** AI runs automatically on every Load & Refresh — no opt-in needed
- Enriches all project descriptions
- Classifies all unmatched chat conversations into initiatives
- **When no API key exists:** A prompt modal appears on Load asking "Enable AI-Enhanced Results?"
- Shows what AI does (enrichment, classification, verification)
- API key input field
- **OK:** saves the key to `~/claude_sync_ai.json`, proceeds with AI enabled
- **Skip:** proceeds without AI

AI Settings modal (manual override):

- API URL (default: `https://api.anthropic.com/v1/messages`)
- API Key (stored locally, masked in UI)
- Model dropdown: Haiku 4.5 (default, cheapest), Sonnet 4.5 (balanced), Opus 4.6 (most capable)

Enrichment modes:

- **Bulk enrichment** — during Load & Refresh background task
- **Per-project enrichment** — "Enrich with AI" button on project detail page

Enrichment prompt:

- Sends: project name, status, progress, category, features, not-working, recent timeline, open threads
- Asks for: 3-5 sentence summary covering what it does, current state, what's happening
- `max_tokens`: 250, `timeout`: 30s

6. PDF Generator (`generate_status_pdf.py`)

Generates a multi-page PDF grouped by category:

Per-project block:

- Header with progress % and status
- Meta line: last active, last synced
- Narrative paragraph
- Recent Activity (last 5 timeline items, human-readable dates)

- Open Threads (max 6, color-coded tags, truncated to 120 chars)

Cross-project section:

- "What's Left Unfinished" — shows all projects with open threads
- Grouped by project name, first 4 threads per project
- Color-coded by thread type

Category ordering: Church, School, Product, Infrastructure, Personal, Research

7. EOL Detection

Projects flagged as "Possibly EOL" if:

- Status contains: abandoned, archived, deprecated, eol, merged, completed, sunset, dead
- No activity in 90+ days
- 0% progress AND no timeline entries

8. Archive & Ignore System

Ignore (for EOL projects on dashboard):

- Ignore button on EOL-flagged cards
- Persisted to `~/ .claudesync_ignored.json`
- Restore button to bring back

Archive (on `/view/projects`):

- Archive button per project
- Archived projects excluded from dashboard stats and chart
- Persisted to `~/ .claudesync_archived.json`
- Unarchive to restore

9. Dev State Tracking

Each project can be assigned a development state reflecting its current work status:

State	Color	Meaning
test	Orange (#E67E22)	Code was pushed with no evidence of testing. Needs testing.
refine	Blue (#3498DB)	User satisfied or riffing/exploring. Chat abandoned in good state.
continue	Red (#E74C3C)	Active testing ongoing but not resolved. Work in progress.

Dashboard display:

- Color-coded badge next to project name on cards and detail page
- Left border accent on project cards matching dev state color
- Dark mode variants for all badge colors

Setting dev state:

- Dropdown on project detail page (test/refine/continue/none)
- Persisted to `PROJECT_STATUS.md` via `| Dev State | value | metadata` row
- If the field doesn't exist in the file, it's auto-inserted after the Status row

AI assessment:

- "AI Assess" button on project detail page

- Sends project timeline + threads to AI
- AI responds with state classification + reasoning
- Auto-selects the suggested state in the dropdown

API endpoints:

- POST /api/dev-state/ — set dev state {dev_state: "test|refine|continue|"}
 - POST /api/ai-assess-state/ — AI classifies dev state, returns {dev_state, reason}

10. Thread Resolution

- Resolve button on threads in detail pages and list views
- Thread key: project_name::thread_text[:100]
- Persisted to ~/.claudesync_resolved.json
- Resolved threads hidden from counts and displays

11. What's Next View (^/view/whats-next`)

Cross-project triage view showing every actionable item across all projects.

Grouped by project, sorted by priority:

- Blockers (red)
- Next Steps (blue)
- Open Questions (orange)
- Not Yet Built (gray)

Per-item actions:

- **Resolve** — marks the thread as resolved
- **Ignore** — dismisses the item, moves to Ignored section with Restore button
- **Reassign** — dropdown of all other projects, moves item to Reassigned section showing "From X → Y", with Undo button

Sections:

- Active items (grouped by project with progress bar and category)
- Reassigned items (with Undo)
- Ignored items (with Restore)

AI Category Verification:

- "Verify All Categories with AI" button at top of page
- Sends all projects to AI in a single call
- Flags low-confidence categorizations inline: red "AI: should be [category]" or green "OK"

Persistence: ~/.claudesync_item_actions.json

12. Unmatched Conversations & Initiative Classification (^/view/unmatched`)

Shows chat conversations that weren't automatically matched to any project by fuzzy name matching.

Stats bar: Unmatched Conversations, New Initiatives Suggested, Matched to Existing, Skipped (small talk)

AI classification (runs automatically during Load & Refresh when key is present):

- Sends unmatched conversations to AI in batches of 15
- AI responds per conversation with:
 - MATCH initiative_name — belongs to an existing project/initiative

- NEW suggested_initiative_name — suggests a new 2-5 word initiative name
- SKIP — too vague, greetings, small talk
- Results grouped by suggested initiative with conversation excerpts
- Can also be triggered manually via "Classify with AI" button

Persistence: ~/.claudesync_initiatives.json

CLI Tools

``init_project_status.py``

```
python3 init_project_status.py ~/myproject --name "My Project" --category School
```

Options: --name (required), --category, --repo, --progress, --status

``update_repos.py``

```
python3 update_repos.py # Auto-detect all
python3 update_repos.py --list # Show current status
python3 update_repos.py --project "Name" --repo "user/repo"
```

``sync_history.py``

```
python3 sync_history.py --assets # Build SHARED_ASSETS.md registry
```

``generate_status_pdf.py`` (standalone CLI)

```
python3 generate_status_pdf.py --scan-paths "~ ~/projects" --chat-history ~/export.zip
```

API Endpoints

Method	Path	Description		
GET	<code>`/`</code>	Dashboard		
GET	<code>`/project/`</code>	Project detail		
GET	<code>`/view/projects`</code>	All projects list		
GET	<code>`/view/events`</code>	All events list		
GET	<code>`/view/threads`</code>	Open threads list		
GET	<code>`/view/blockers`</code>	Blockers list		
GET	<code>`/view/whats-next`</code>	Cross-project triage view		
GET	<code>`/view/unmatched`</code>	Unmatched conversations + AI classification		
GET	<code>`/api/progress`</code>	Background task status (running, step, detail)		
GET	<code>`/api/chart-data?mode=day\`</code>	week\`	month`	Activity chart data
POST	<code>`/api/ignore`</code>	Hide project <code>`{name}`</code>		
POST	<code>`/api/restore`</code>	Unhide project <code>`{name}`</code>		
POST	<code>`/api/archive`</code>	Archive project <code>`{name}`</code>		
POST	<code>`/api/unarchive`</code>	Unarchive project <code>`{name}`</code>		

POST	`/api/resolve-thread`	Resolve thread `{project, text}`		
POST	`/api/item-action`	Ignore/reassign/undo item `{key, action, target_project?}`		
POST	`/api/verify-categories`	AI-verify all project categories		
POST	`/api/classify-unmatched`	AI-classify unmatched chat conversations		
POST	`/api/enrich/`	AI-enrich single project		
POST	`/api/dev-state/`	Set project dev state (test/refine/continue)		
POST	`/api/ai-assess-state/`	AI-classify project dev state		
GET/POST	`/api/ai-config`	Get/save AI configuration		
POST	`/upload`	Upload chat zip, start background processing (returns JSON)		
GET	`/generate-pdf`	Download PDF report		

Data Model

Project Object

```
{
  'name': str,
  'file_path': str, # Path to PROJECT_STATUS.md
  'project_path': str, # Project directory
  'repo': str, # GitHub user/repo
  'category': str, # Raw category
  'category_group': str, # Normalized (Church, School, Product, etc.)
  'progress': int, # 0-100
  'status': str, # Not Started, In Progress, Beta, Complete, etc.
  'dev_state': str, # test, refine, continue, or empty
  'has_repo': str, # Yes/No
  'has_github_repo': bool, # Computed
  'last_worked': str, # YYYY-MM-DD
  'last_synced': str, # YYYY-MM-DD
  'summary': str, # Description from status file
  'narrative': str, # AI-generated or summary-derived
  'features': [str], # What's Working items
  'not_working': [str], # What's Not Working items
  'blockers': [str], # Blockers
  'next_steps': [str], # Next Steps
  'open_questions': [str], # Design Questions from tables
  'progress_log': [dict], # Dated entries from Progress Log section
  'git_commits': [dict], # Recent commits from git history
  'timeline': [dict], # Merged timeline (progress + git + chat)
  'open_threads': [(str,str)], # (type, text) – all unfinished items
  'chat_conversations': int, # Number of chat files that mentioned this project
}
```

Timeline Entry

```
{'date': 'YYYY-MM-DD', 'source': str, 'text': str}
```

Thread Types

- **Blocker** (red) — from Blockers section
- **Open Question** (orange) — from Design Questions table
- **Next Step** (blue) — from Next Steps section
- **Not Yet Built** (gray) — from What's Not Working section

Background Task State

```
{
  'running': bool,
  'step': str, # scanning, parsing, reading_chat, matching_chat, ai_enrichment, ai_classify, building,
  done
  'detail': str, # Human-readable progress message
  'error': str, # Error message if failed
  'result_msg': str, # Summary message on completion
}
```

PROJECT_STATUS.md Schema

```
# PROJECT_STATUS: [Project Name]

## Metadata
| Field | Value |
|-----|-----|
| **Project Name** | [Name] |
| **Repository** | [user/repo or "(not created)"] |
| **Category** | [Infrastructure/School/Church/Product/Research/Personal] |
| **Progress** | [0-100]% |
| **Status** | [Not Started/In Progress/Beta/Complete/Shelved] |
| **Last Worked** | [YYYY-MM-DD] |
| **Has GitHub Repo** | [Yes/No] |
| **Last Synced to Claude.ai** | [YYYY-MM-DD] |
| **Dev State** | [test/refine/continue] |

## Current State
### What's Working
### What's Not Working
### Blockers

## Progress Log
### YYYY-MM-DD (Source Description)

## Open Design Questions
| Question | Status | Notes |

## Next Steps

## Reusable Assets
| Asset | Description | Tags |
```

Progress Color Coding

Progress	Color
> 75%	Green (#2ECC71)
50-75%	Blue (#3498DB)

25-49%	Yellow (#F1C40F)
< 25%	Red (#E74C3C)

Dev State Color Coding

State	Color	Badge	Card Border
test	Orange (#E67E22)	Orange bg/text	Left orange border
refine	Blue (#3498DB)	Blue bg/text	Left blue border
continue	Red (#E74C3C)	Red bg/text	Left red border

Theming

Light and dark themes via CSS custom properties on [data-theme]. Persisted in localStorage, applied before paint to prevent flash. Toggle button in top bar on every page.

Dependencies

Python

```
flask
reportlab>=4.0.0
matplotlib>=3.7.0
numpy>=1.24.0
```

System

- Python 3.8+
- Git (commit history + remote detection)
- macOS or Linux

Optional

- Anthropic API key (for AI enrichment + classification)

File Structure

```
~/claudesync2/
■■■■ web_app.py # Flask dashboard (2652 lines)
■■■■ generate_status_pdf.py # Scanner, parser, chat matcher, PDF renderer (1238 lines)
■■■■ init_project_status.py # Project initializer CLI
■■■■ update_repos.py # Batch repo updater CLI
■■■■ sync_history.py # Asset registry builder
■■■■ requirements.txt # Python dependencies
■■■■ .gitignore
■■■■ CLAUDE.md # Claude Code behavioral instructions
■■■■ PRODUCT_SPEC.md # This document
■■■■ PROJECT_STATUS.md # This project's own status
■■■■ README.md
■■■■ templates/
```

```
■ ■■■ CLAUDE.md # Template for new projects
■ ■■■ PROJECT_STATUS.md # Template for new projects
■■■ daily_status_report.sh # Shell script for cron/launchd
■■■ com.ctreada.dailystatusreport.plist # macOS LaunchAgent
```

Persistence Files (in `~/.`)

```
~/.claudesync_ignored.json # Hidden project names
~/.claudesync_resolved.json # Resolved thread keys
~/.claudesync_archived.json # Archived project names
~/.claudesync_ai.json # AI API config (url, key, model)
~/.claudesync_import_meta.json # Last chat import metadata
~/.claudesync_item_actions.json # Item ignore/reassign decisions
~/.claudesync_initiatives.json # AI classification of unmatched chats
```

Quick Start

```
# Install
cd ~/claudesync2
pip3 install -r requirements.txt

# Run the dashboard
python3 web_app.py
# Open http://localhost:5111

# Or generate PDF from CLI
python3 generate_status_pdf.py --scan-paths ~ --chat-history ~/export.zip
```

Out of Scope (For Now)

Automatic Claude.ai upload — user must manually upload PDF

Real-time sync — PDF/dashboard is point-in-time snapshot

Progress auto-calculation — user sets progress % manually

Multi-user support — single developer only

Cloud storage — all data is local, nothing leaves the machine

Windows support — untested

Last updated: 2026-02-11

Version: 7.0